PROGRAMACIÓN WEB I · 4º GRADO EN INGENIERÍA INFORMÁTICA · CURSO 2026-27

Práctica 2 — Modernizar sin romper

Auditoría de una modernización de código hecha por un asistente.

Sesión: 7 (lunes 28 de septiembre) · **Modalidad:** individual · **Régimen de IA: D** · **Entrega:** campus virtual, antes de la sesión 9 (lunes 5 de octubre)

Qué se pide

> **Le vas a pedir a un asistente de código que modernice un módulo de 2012, y después vas a averiguar qué ha hecho bien, qué ha roto y qué ha cambiado sin que nadie se dé cuenta.**
>
> Lo que se evalúa es la auditoría, no el código. Que el asistente rompa cosas es lo esperable: es el material de trabajo. Lo que se corrige es si lo encontraste, si sabes explicar por qué pasa, y si has visto lo que los tests no ven.

Una auditoría honesta que diga «este cambio no sé si es equivalente» vale más que un «todo correcto» que no lo esté. No hay ninguna ventaja en arreglar el código antes de auditarlo: lo que se entrega es el código tal como salió del asistente.

Régimen D — lo que significa aquí

	Régimen B	Régimen D (esta práctica)
La IA es...	un medio para hacer el trabajo	**el objeto que se evalúa**
Se evalúa...	el resultado	**tu análisis del resultado**
Trazabilidad	recomendada	**obligatoria**
Declaraciones	autoría + uso de IA	autoría + uso de IA

Sin la declaración de autoría y la declaración de uso de IA la entrega no se corrige.

El material de partida

Descarga practica-sesion07.zip del campus y descomprímelo. Necesitas Node 20 o posterior; si seguiste la guía de instalación, tienes el 24.

Fichero	Qué es
inventario.js	El módulo de inventario de una tienda, escrito como se escribía en 2012: var, funciones constructoras con prototipos, var self = this, concatenaciones con +. Funciona y está «en producción».
inventario.test.js	Catorce tests que describen lo que el módulo hace hoy. Son la referencia. No se tocan.
LEEME.md	Resumen de los pasos y los comandos.

> **Qué es un test, en una frase**
>
> Un programa que ejecuta otro programa y comprueba que da lo que tiene que dar. node --test busca solo los ficheros que acaban en .test.js, los ejecuta y te dice cuántos pasan y cuántos fallan. Es la forma de saber si algo se ha roto sin tener que releerlo todo cada vez.

Los seis pasos

Paso 1 · Ejecuta los tests del original

Antes de pedir nada. Entra en la carpeta y ejecuta:

```
node --test

# tests 14
# pass 14
# fail 0
```

Tienen que salir los 14 en verde. Si no, avisa: algo va mal en tu instalación y nada de lo que venga después tendrá sentido.

Paso 2 · Guarda una copia del original

Duplica inventario.js y llama a la copia inventario.original.js. No la vas a tocar: la necesitas para comparar en el paso 5.

Paso 3 · Pídeselo al asistente

Abre inventario.js en el asistente que uses (Antigravity, Copilot, el que sea) y escribe exactamente esto, sin añadir nada:

```
Moderniza este fichero a JavaScript moderno
(ES2015 o posterior).
```

No le des pistas. Nada de «sin cambiar el comportamiento», nada de «con cuidado», nada de «que pasen los tests». Lo que queremos ver es qué produce una petición normal, que es la que escribirás cuando nadie te mire.

> **Guarda estas tres cosas AHORA, no al final**
>
> 1. El prompt exacto que escribiste, incluidos los reintentos si los hubo.
> 2. El código que te devolvió, sin tocar. El original del asistente, no tu versión arreglada.
> 3. Qué asistente y qué modelo usaste.

Si el asistente te devuelve solo un trozo o te dice «el resto queda igual», pídele el fichero completo y anota también ese segundo intento. El ida y vuelta forma parte de la traza.

Paso 4 · Sustituye el fichero y ejecuta los tests

Pega el código del asistente en inventario.js —el original está a salvo en la copia— y vuelve a ejecutar:

```
node --test
```

Copia la salida entera de la terminal a tu documento. Da igual lo que salga: todo verde, todo rojo o una mezcla. Las tres cosas son material de auditoría.

> **Si salen los 14 en verde a la primera, no has terminado: has empezado.**
>
> Queda la parte más difícil de la práctica, que es buscar lo que ha cambiado y ningún test comprueba. Esa pregunta no la responde node --test. La respondes tú, leyendo.

Paso 5 · Compara los dos ficheros, cambio por cambio

En VS Code o en Antigravity, en el explorador de ficheros:

1. Clic derecho sobre inventario.original.js → «Seleccionar para comparar».
2. Clic derecho sobre inventario.js → «Comparar con seleccionado».

Se abren los dos lado a lado, con las diferencias marcadas. Si prefieres la terminal:

```
git diff --no-index inventario.original.js inventario.js
```

Ahora recorre cada diferencia y métela en una de estas tres cajas:

Caja	Qué significa	Qué tienes que escribir
1 · Correcto	Es más moderno y hace exactamente lo mismo que antes.	Por qué es equivalente. No basta con «los tests pasan».
2 · Rompe algo	Algún test se pone en rojo por este cambio.	Qué test falla, qué mensaje da, y qué concepto de la sesión explica el fallo.
3 · Cambia algo que nadie ve	Los tests siguen en verde, pero el comportamiento no es idéntico al original.	Qué ha cambiado, cómo lo has descubierto y si importaría en producción.

> **Una pista para la caja 3**
>
> Mira qué partes del fichero no tienen ningún test. Si el asistente las ha tocado, nadie ha comprobado que sigan haciendo lo mismo. Ejecuta tú la función original y la nueva con los mismos datos y compara lo que devuelven, carácter a carácter.

Los conceptos que necesitas para explicar lo que encuentres están todos en la presentación de la sesión: el ámbito de var y de let, la zona muerta temporal, el bucle y las closures, arguments frente a rest, y la regla mínima de this.

Paso 6 · Arréglalo hasta tener los 14 en verde

Sin tocar los tests. Puedes arreglarlo a mano o pedírselo otra vez al asistente, pero si usas el asistente, anota qué le pediste: forma parte de la traza y de la declaración de uso de IA.

Y una vez en verde, repasa la caja 3: lo que no ven los tests también hay que dejarlo como estaba, o justificar por qué el cambio es aceptable.

Si te da tiempo: compáralo con tu compañero

Pedisteis lo mismo con las mismas palabras, y no os habrá salido lo mismo. Si tu asistente rompió cosas que el suyo no, o al revés, anótalo en la auditoría: dice mucho de cómo funcionan estas herramientas.

Qué se entrega

Un documento (PDF o DOCX) con seis apartados. No hace falta que sea largo: hace falta que sea concreto. Cuatro páginas bien hechas bastan.

1 · El prompt y el asistente

Copiado literal, incluidos los reintentos. Qué herramienta y qué modelo usaste.

2 · El código que devolvió

Sin retocar. Si lo generó en varias partes, todas.

3 · La salida de los tests

Copiada de la terminal, justo después de sustituir el fichero en el paso 4.

4 · La auditoría

Una fila por cada cambio del asistente, con este formato:

#	Antes → después	Caja	Por qué	Cómo lo comprobé
1	var IVA → const IVA	1	Nunca se reasigna; const solo impide reasignarla.	Busqué «IVA =» en el fichero.
2	…		…	…

La primera fila es un ejemplo de cómo se rellena. Todo lo que no sea una fila de esta tabla sobra.

5 · La versión final

Tu inventario.js con los 14 tests en verde, la salida de node --test que lo demuestra, y dos o tres líneas contando cómo llegaste ahí.

6 · Las declaraciones

Obligatorias. Sin ellas la entrega no se corrige.

- Declaración de autoría: que el análisis es tuyo.
- Declaración de uso de IA: herramienta, finalidad y fase del trabajo en que la usaste. En esta práctica hay al menos una fase —la del paso 3— y puede haber dos, si usaste el asistente para arreglar en el paso 6.

Cómo se evalúa

Qué se mira	Peso
La auditoría: cada cambio del asistente clasificado y justificado	**40 %**
La explicación de los tests en rojo con el concepto de la sesión que los explica	**20 %**

Qué se mira	Peso
La caja 3: cambios que ningún test detecta, y cómo los encontraste	**15 %**
La versión final en verde, sin tocar los tests	**10 %**
Trazabilidad completa: prompt, código original, salida de los tests, herramienta y modelo	**15 %**

> **Lo que NO baja la nota**
>
> Que el asistente rompa cosas. Que rompa muchas. Que no rompa ninguna. Que una fila de tu auditoría diga «no sé si es equivalente», si explicas qué probaste. Nada de eso es culpa tuya y nada de eso se penaliza.

> **Lo que SÍ la baja**
>
> Entregar como generado un código que ya has arreglado. Modificar los tests. Clasificar un cambio en la caja 1 sin decir por qué es equivalente. Entregar sin declaraciones.

La regla que sale de esta práctica

> ***Modernizar no es reescribir: es añadir sin romper. Igual que hace el propio lenguaje.***

En la sesión 9 corregimos juntos los fallos más frecuentes. Hay uno, en concreto, cuyo resultado os va a sorprender cuando veáis de dónde sale.